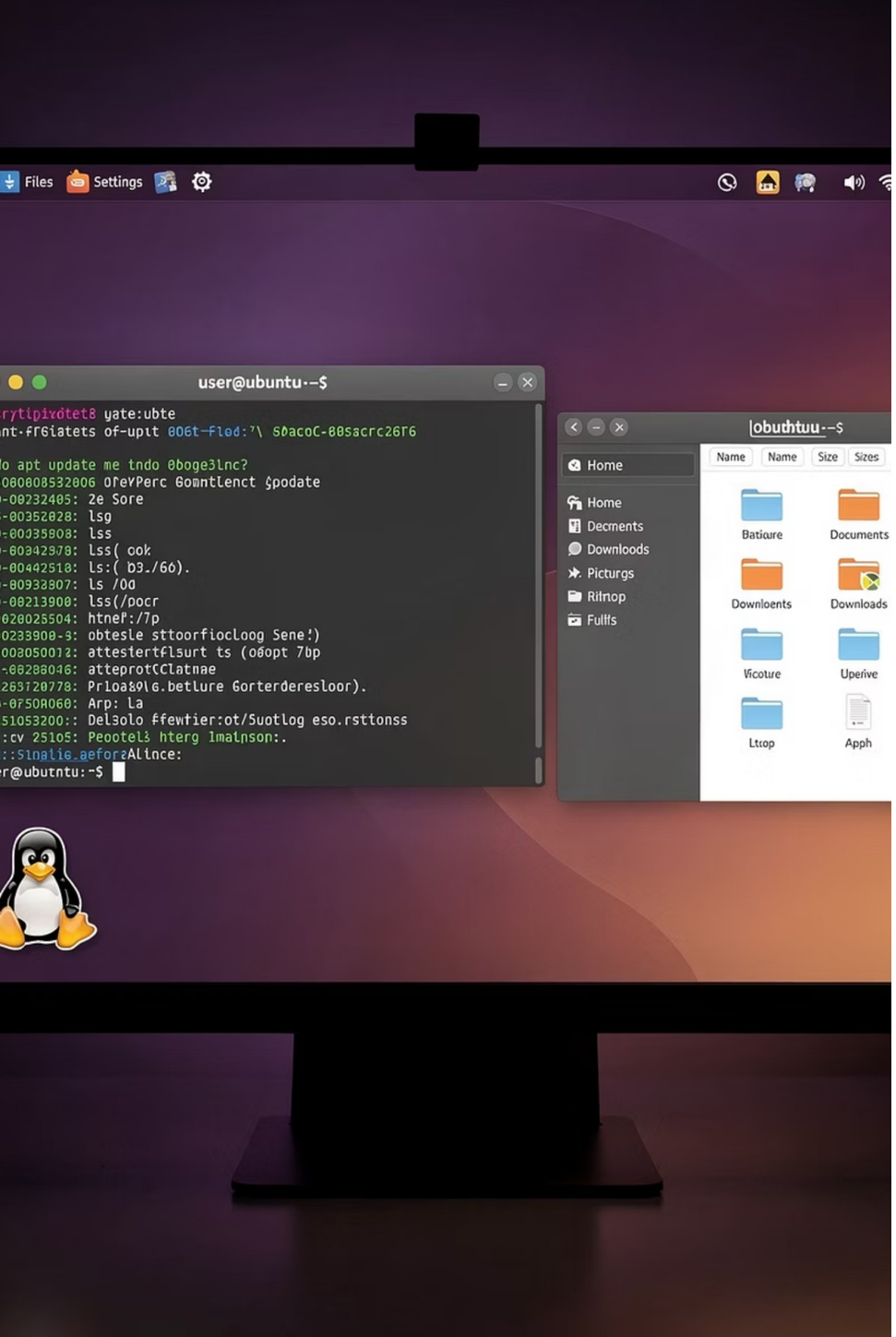


Sistemi Operativi

UNIX · Linux · Bash — Primi Comandi

Prof. Fedeli Massimo - Fabbrica Digitale – Tutti I diritti riservati

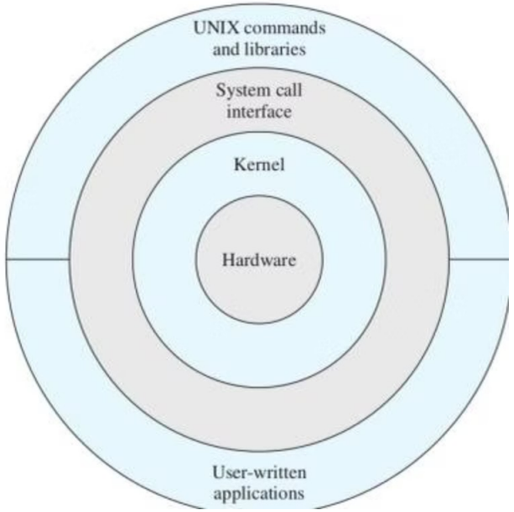
Il Sistema Operativo UNIX

Origini

UNIX viene progettato negli anni '70 presso i Bell Labs. Per la prima volta un sistema operativo viene scritto in un linguaggio ad alto livello — il **C** — invece che in assembly.

Ancora oggi quasi tutte le implementazioni di UNIX sono scritte in C.

Architettura di UNIX



Struttura a Strati

Nell'architettura di UNIX l'hardware è completamente circondato dal SO. Il SO viene chiamato **kernel** perché isolato da utente e applicazioni. UNIX comprende:

- Shell e interfaccia software
- Componenti del compilatore C (loader, assembler, compiler)
- Strato esterno con applicazioni utente e interfaccia compilatore C

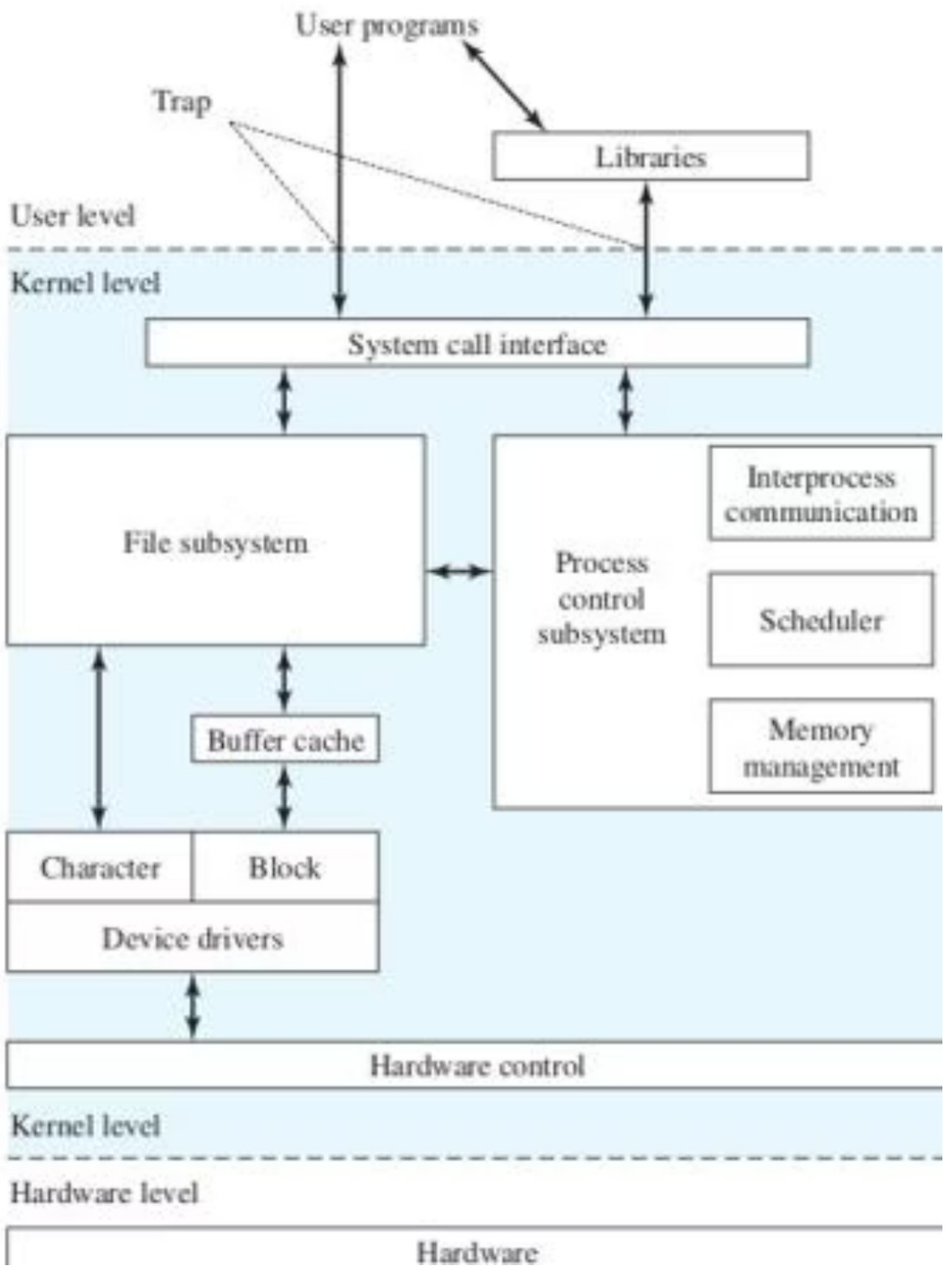
UNIX

Schema del Kernel

I **programmi utente** invocano i servizi del SO direttamente o tramite librerie.

L'interfaccia di chiamate al sistema rappresenta il confine con l'utente e permette di accedere a funzioni specifiche del kernel tramite software di livello più alto.

Il SO contiene routine che interagiscono direttamente con l'hardware.



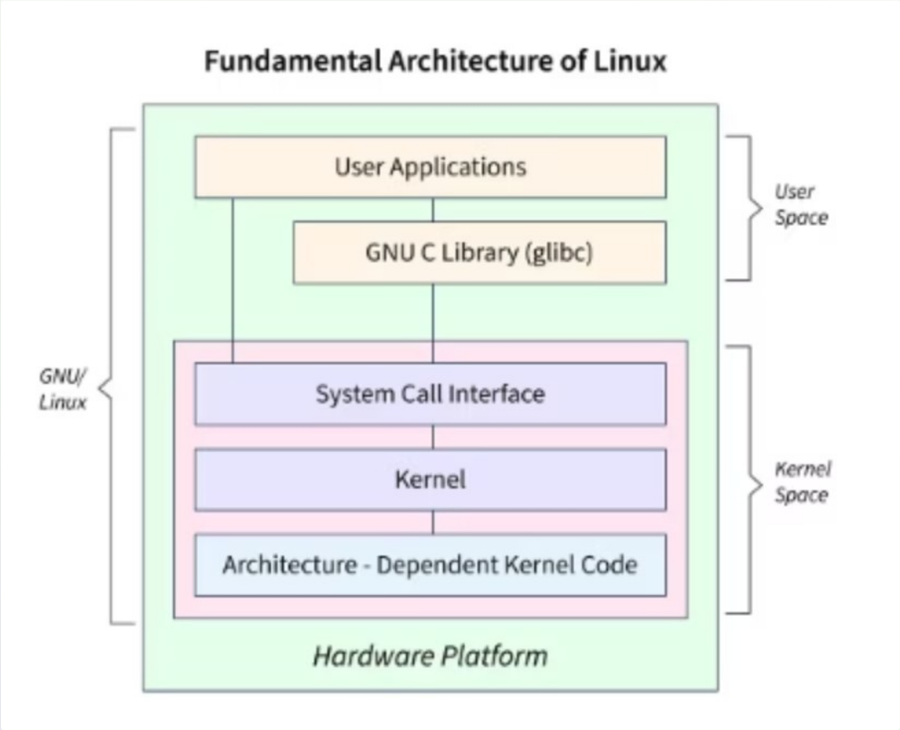
UNIX Tradizionale

Due Parti del Kernel

Lo strato tra le due interfacce è diviso in:

Controllo di sistema: gestione della memoria, assegnazione e sincronizzazione dei processi, comunicazione tra processi

Gestione file e I/O: il file-system scambia dati tra memoria e dispositivi esterni



UNIX

Limiti del UNIX Tradizionale

Singolo Processore

Progettato per un singolo processore, senza protezione delle strutture dati da accessi concorrenti

Kernel Rigido

Supporta **un solo tipo di file system**, una sola politica di scheduling e un solo formato di eseguibili

Kernel Sproporzionato

L'aggiunta di nuove capacità ha prodotto un kernel ridondante e non modulare

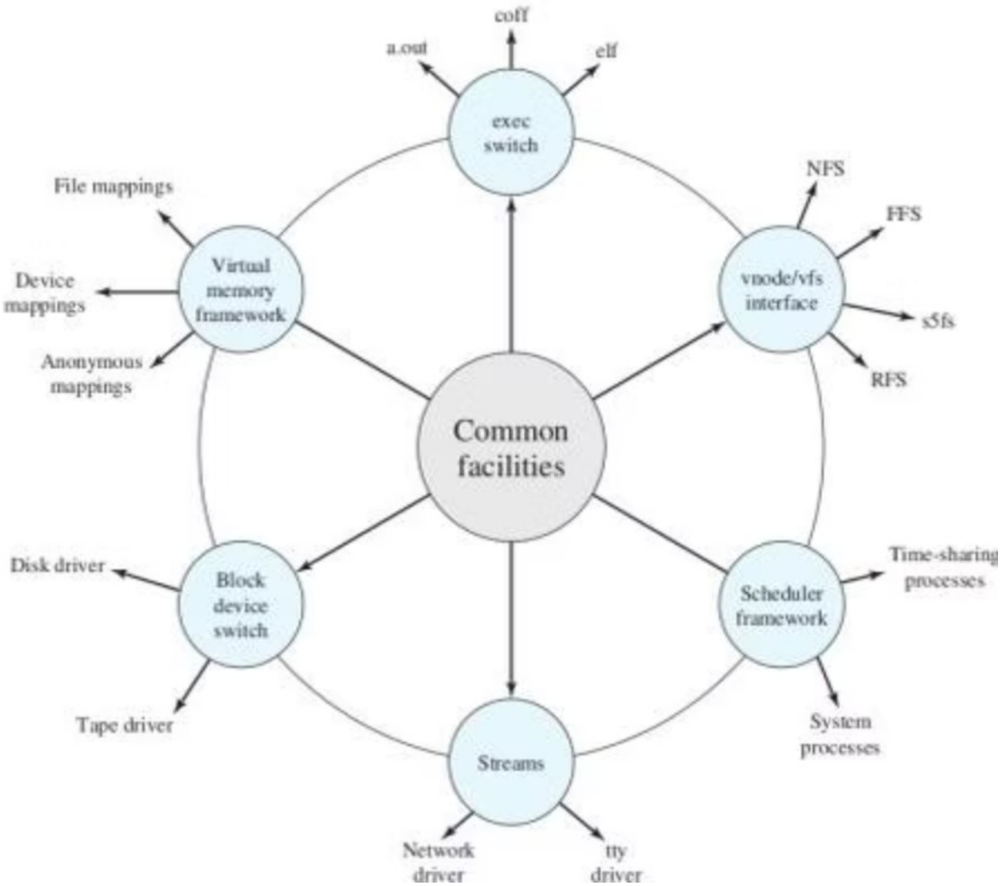
UNIX Moderno: Architettura Modulare

Perché l'Evoluzione?

- Necessità di unificare le molte innovazioni introdotte
- Aggiungere caratteristiche più attuali

Produrre un'architettura più **modulare**

Un piccolo **core** di funzioni e servizi scritti in maniera modulare è necessario ai processi del SO. Le funzioni nei cerchi esterni possono essere implementate in una varietà di modi.



Kernel Monolitico vs Microkernel

Kernel Monolitico

- Tutte le funzionalità in un unico blocco di codice
- Kernel tutto in memoria dal boot allo spegnimento
- Cambiare una porzione richiede rilinkare e reinstallare tutto
- Più efficiente in velocità, ma occupa più memoria

Microkernel

- Solo una minima parte del kernel è sempre in memoria
- Sempre in memoria: scheduler, sincronizzazione
- Solo a richiesta: gestore memoria, filesystem, driver

Eccezione degna di nota: **Mac OS X**

Quasi tutti i sistemi operativi moderni sono a **kernel monolitico**.

Kernel Monolitico

Nel **kernel monolitico** tutti i servizi del sistema operativo vengono eseguiti in **kernel space**:

- gestione memoria
- scheduler
- filesystem
- driver
- rete
- gestione processi

Tutti questi componenti fanno parte di un unico grande programma e comunicano direttamente tra loro tramite chiamate a funzione.

Kernel Monolitico

Vantaggi:

- Prestazioni molto elevate
- Comunicazione interna molto veloce

Svantaggi:

- Se un driver va in errore → crash dell'intero sistema
- Kernel molto grande e complesso
- Minore sicurezza

Esempi:

- Linux
- Unix
- BSD

Microkernel

Nel **microkernel** nel kernel rimangono solo le funzioni essenziali:

- gestione processi
- gestione memoria
- comunicazione tra processi (IPC)

Tutti gli altri servizi (driver, filesystem, rete) vengono eseguiti in **user space** come processi separati.

La comunicazione avviene tramite **message passing (IPC)**.

Microkernel

Vantaggi:

- Sistema molto stabile (se un driver si blocca non cade il sistema)
- Maggiore sicurezza
- Architettura modulare e più facile da mantenere

Svantaggi:

- Più lento (comunicazione tramite messaggi)
- Maggiore overhead

Esempi:

- MINIX
- Mach
- QNX
- Parte di MacOS (XNU è ibrido)
- Windows NT (kernel ibrido)

Monolitico Modulare

Un **kernel monolitico modulare** è un kernel **monolitico** (cioè con tutti i servizi principali che girano in kernel space) ma progettato in modo da poter **caricare e scaricare dinamicamente dei moduli** (driver, filesystem, ecc.) senza dover ricompilare o riavviare il kernel.

In altre parole:

Monolitico → tutto gira nel kernel space
Modulare → alcune parti possono essere aggiunte o rimosse come moduli

Il caso tipico è **Linux**.

Monolitico Modulare

Parte fissa (kernel base):

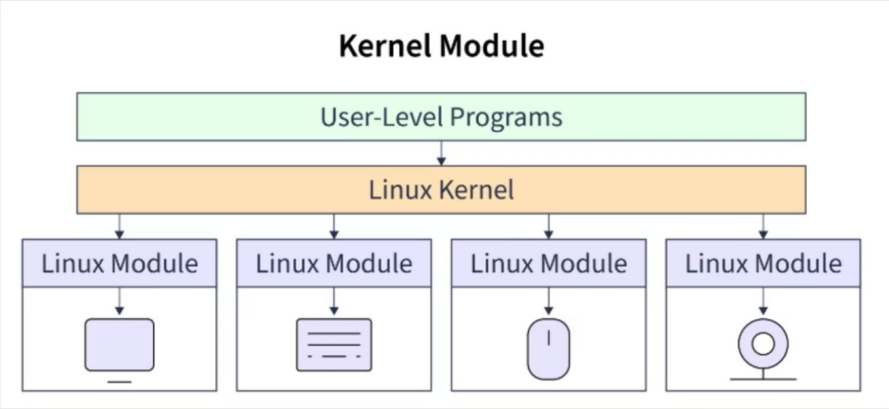
- scheduler
- gestione memoria
- gestione processi
- IPC
- parte base del filesystem
- networking base

Moduli caricabili (LKM – Loadable Kernel Modules):

- driver (stampanti, schede video, rete…)
- filesystem (es. ext4, ntfs, fat32)
- moduli di rete
- moduli di sicurezza

I moduli vengono caricati con comandi tipo:

`lsmod` → mostra i moduli caricati
`modprobe` → carica un modulo
`rmmod` → rimuove un modulo



Ibrido

Un **kernel ibrido** è un kernel che nasce come **microkernel**, ma sposta **alcuni servizi di nuovo nel kernel space** per migliorare le prestazioni. In pratica è un compromesso tra **monolitico** e **microkernel**.

- Il microkernel puro è molto sicuro ma lento (troppi messaggi IPC).
- Il monolitico è molto veloce ma meno sicuro.

L'ibrido cerca di ottenere **velocità del monolitico + struttura del microkernel**.

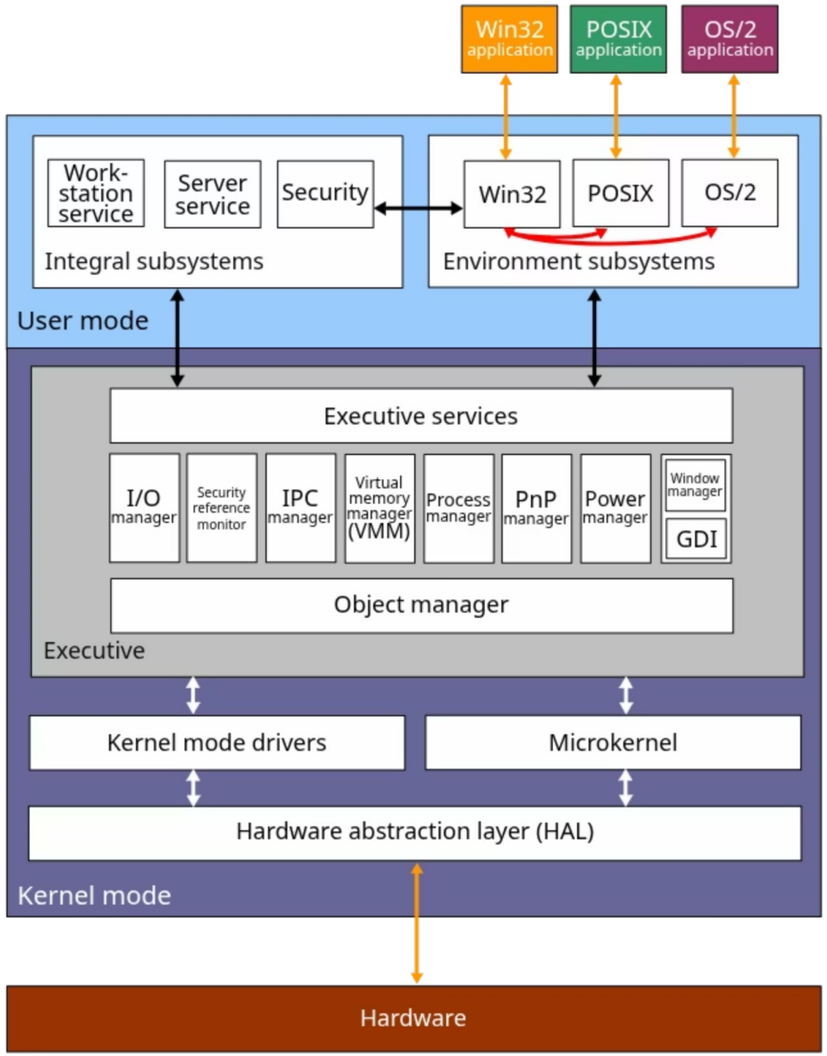
Ibrido

Nel kernel space:

- scheduler
- memoria
- IPC
- parte dello stack di rete
- filesystem
- alcuni driver

In user space:

- altri driver
- servizi
- sottosistemi



Monolitico modulare vs Ibrido

Nel **monolitico modulare**:

Tutti i servizi del sistema operativo girano **in kernel space**.
Però possono essere caricati come **moduli dinamici** (driver, filesystem, rete...).
I moduli **girano comunque in kernel mode**.

Es. Linux

Nel **kernel ibrido**:

Alcuni servizi stanno **in kernel space**
Altri servizi stanno **in user space**

- Struttura derivata dal microkernel, ma con parti riportate nel kernel per velocità

Es. Windows NT, MacOs

Panoramica dei sistemi operativi

Sistema Operativo	Tipo
Linux	Monolitico modulare
Windows NT	Ibrido
MacOS	Ibrido (Mach + BSD)
MINIX	Microkernel
QNX	Microkernel

```
fedeli@fedora:~$ ls -l
totale 896716
drwxr-xr-x. 1 fedeli fedeli         6  3 ott 21.03 Android
drwxr-xr-x. 1 fedeli fedeli      1058 20 mar 08.45 Documenti
drwxr-xr-x. 1 fedeli fedeli        24  7 ott 09.53 eclipse
drwxr-xr-x. 1 fedeli fedeli        34 21 ott 21.34 eclipse-workspace
drwxr-xr-x. 1 fedeli fedeli       510 12 feb 21.47 Immagini
drwxr-xr-x. 1 fedeli fedeli         0 12 set  2025 Modelli
drwxr-xr-x. 1 fedeli fedeli        82  3 nov 22.28 Musica
drwxr-xr-x  1 fedeli fedeli        16 21 ott 21.50 NetBeansProjects
drwxr-xr-x  1 fedeli fedeli         0 18 mar 20.57 PacketTracer
drwxr-xr-x  1 fedeli fedeli        10 11 mar 17.07 Postman
drwxr-xr-x. 1 fedeli fedeli         0 12 set  2025 Pubblici
drwxr-xr-x. 1 fedeli fedeli        68 27 mar 09.00 Scaricati
drwxr-xr-x. 1 fedeli fedeli       962 16 feb 18.31 Scrivania
drwxr-xr-x. 1 fedeli fedeli        54  5 ott 21.22 StudioProjects
-rw-r--r--  1 fedeli fedeli 128586881 30 set 13.34 teamviewer.x86_64.rpm
drwxr-xr-x  1 fedeli fedeli       344  3 nov 17.18 TornelloSync
drwxr-xr-x. 1 fedeli fedeli        48 21 gen 08.36 Video
-rw-r--r--  1 fedeli fedeli 789645312 12 set  2025 virtio-win.iso
drwxr-xr-x  1 fedeli fedeli        40  4 nov 17.43 whisper
fedeli@fedora:~$
```

CARATTERISTICHE UNIX

Caratteristiche Principali di Unix

Caratteristiche Fondamentali

Multiutente

Più utenti possono usare contemporaneamente lo stesso computer

Multiprocesso

Lo stesso utente può lanciare contemporaneamente più di un processo

File System Gerarchico

Organizzato come un albero: nodi interni sono directory, foglie sono file o directory vuote

File System Aggiuntivi

È possibile aggiungere file system aggiuntivi (es. USB, CD) montandoli su una directory

Componenti del Sistema Unix

Kernel

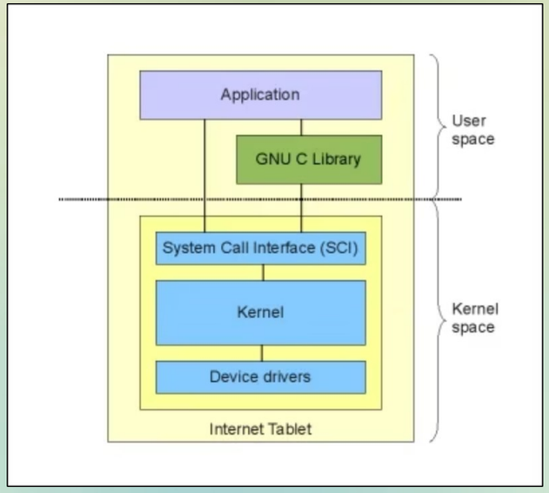
Gestisce memoria (principale e secondaria), processi, I/O e risorse hardware in generale

System Calls

Funzioni C che possono essere chiamate per interfacciarsi con il kernel (es. per creare un file)

Shell

Programma interattivo che accetta comandi da girare al kernel (es. mostrare il contenuto di una directory)



Ambienti di Programmazione e Utilities

Ambienti di Programmazione

Per scrivere programmi, tipicamente in C: compilatore, debugger, text editor. I linguaggi interpretati (Python, Java) richiedono un interprete come intermediario. Il **C** è il linguaggio principe per dialogare direttamente con il kernel.

Utilities

- Suite Office (OpenOffice, LibreOffice)
- Browser (Firefox, Chrome)
- Riproduttore audio/video (VLC)
- Editor di immagini (GIMP)

Modularità: programmi installabili a pacchetti, moduli del SO attivabili e disattivabili.

LINUX

Linux

Il kernel moderno, le distribuzioni e come installarlo

LINUX

Kernel Moderno di Linux

Architettura a Moduli

Linux è principalmente monolitico, ma ha i **moduli**, con vantaggi simili ai microkernel. È strutturato come una collezione di moduli che possono essere aggiunti e tolti a richiesta: **loadable modules**.

Esempi: diversi file system, driver per dispositivi I/O, funzionalità di rete. Un modulo viene eseguito in *kernel mode* dal processo corrente.

Moltissime Versioni

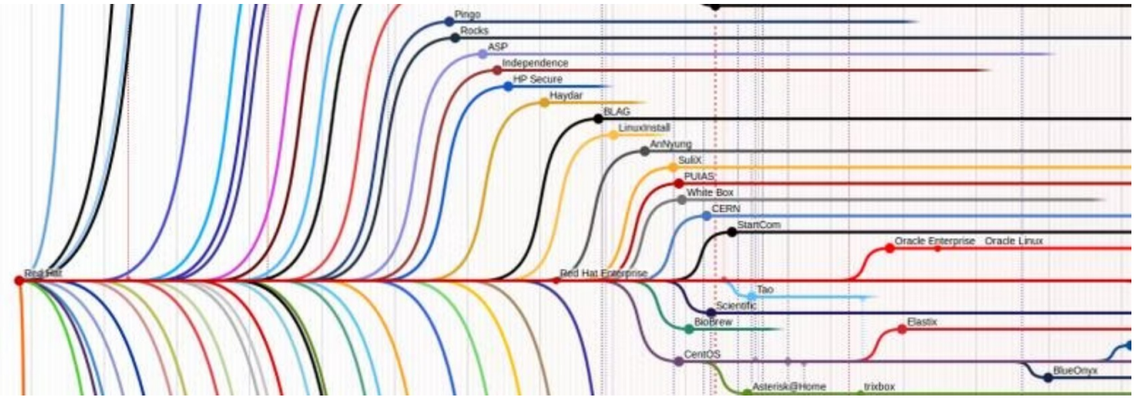
Esistono innumerevoli distribuzioni Linux.

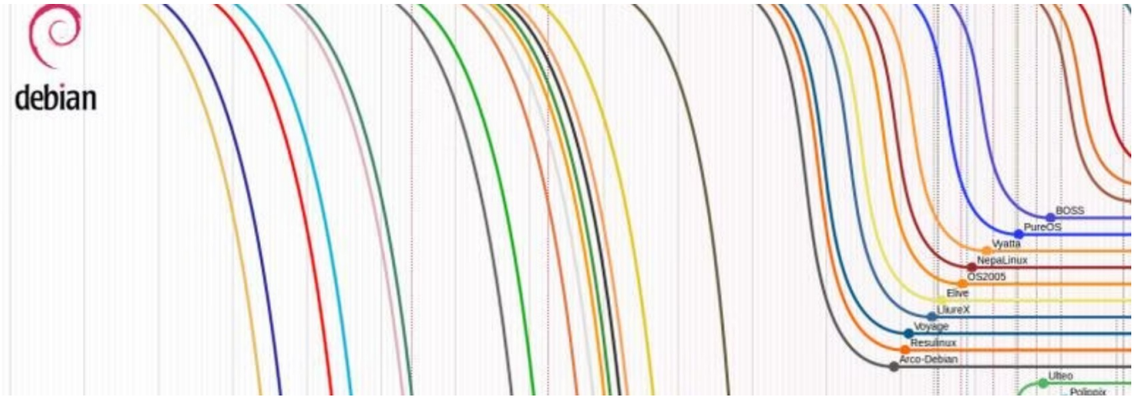
La **timeline** delle distribuzioni (da Wikipedia) mostra la complessità dell'ecosistema Linux dal 1991 ad oggi.

LINUX

Timeline delle Distribuzioni Linux

Particolare della timeline delle distribuzioni di Linux (da Wikipedia): le linee colorate rappresentano le diverse distribuzioni che si diramano nel tempo.





LINUX

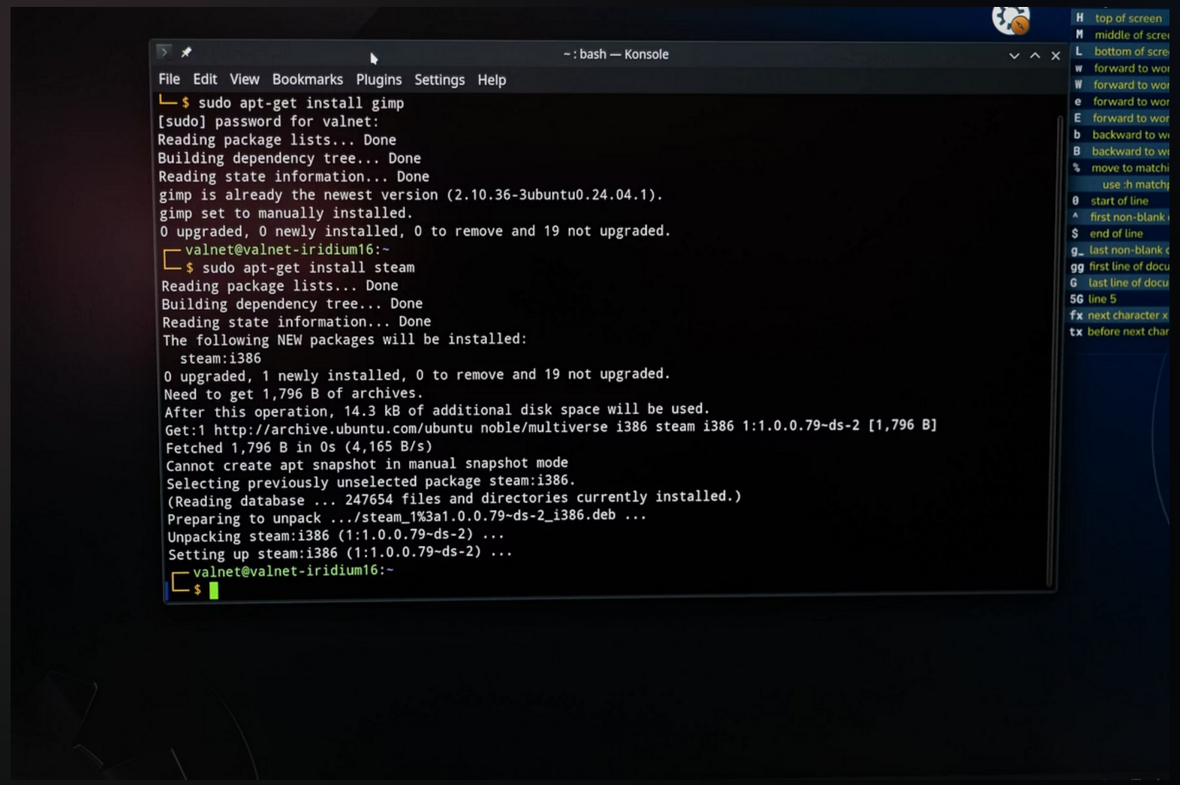
Installare Linux: Macchina Virtuale e WSL

Macchina Virtuale (VirtualBox)

Un applicativo che replica in tutto e per tutto un computer, a partire dal tasto di accensione. La rete può essere condivisa con il sistema operativo *ospitante*. Il SO eseguito dentro VirtualBox è detto *ospitato*.

WSL – Windows Subsystem for Linux

Funzionalità di Windows che consente di eseguire un file system Linux, insieme agli strumenti da riga di comando e alle app GUI di Linux, direttamente in Windows, insieme alle tradizionali app Windows.



BASH

Introduzione alla Bash

Bourne Again Shell – l'interfaccia tra utente e sistema operativo

BASH

Terminale e Shell

Come Funziona

Per interagire con il SO si danno comandi dalla riga di comando usando il **terminale**. Quando si apre il terminale viene avviata la **shell standard**, come la *Bash (Bourne Again Shell)*.

Funzioni della Shell

- Comprende un interprete della riga di comando
- Accoglie gli input utente tramite tastiera
- Avvia i programmi e restituisce il risultato come testo

BASH

Bash Shell: Comandi Utili

Identificare la Shell

`echo $0` oppure `ps -p "$$" -ocmd -h`

History dei Comandi

Il comando `history` mostra i comandi eseguiti. Si replicano con i tasti freccia ↑ ↓. Si ricercano parti di comandi con `CTRL+r`.

Il Prompt

Formato tipico: `nomeutente@nomeemacchina:~cammino$`. La tilde `~` indica la home. Se la cwd non è nel sottoalbero della home, appare il path assoluto.

BASH

Standard Input, Output ed Error

stdin (0)

Standard input stream: accetta testo come input dall'utente

stdout (1)

Standard output stream: testo prodotto in output dal comando, scritto sullo schermo

stderr (2)

Standard error stream: messaggi di errore visualizzati sullo schermo

Come per i file, agli standard stream vengono associati degli identificatori numerici: **0** per stdin, **1** per stdout e **2** per stderr.

PER COMINCIARE

I Comandi

Sintassi, opzioni e convenzioni per l'uso dei comandi in Linux

COMANDI

Sintassi dei Comandi

> `comando [opzioni] argomenti_obbligatori`

Parentesi Quadre [ ]

Tutto ciò che è tra parentesi quadre può essere omesso. Esempio: `cp [-r] [-i] {file sorgenti} filedestinazione`

Puntini ...

Con parentesi quadre e puntini ci possono essere 0, 1 o più argomenti. Esempio: `ps [opzioni] [pid...]`

Parentesi Graffe { }

Deve esserci almeno un argomento. Esempio: `chmod mode[,mode...] filename`

COMANDI

Le Opzioni dei Comandi

Formato delle Opzioni

Un dash: un solo carattere (versione *vecchia*): `-r`

Due dash: una parola (versione *moderna*): `--recursive`

Spesso equivalenti: `-i` e `--interactive`

Con o Senza Argomento

Senza argomento: `-r`, `--recursive`

Con argomento: `-k1`, `-k 1`, `--key=1`

Le opzioni senza argomento con un trattino sono raggruppabili: `-b -r -c` equivale a `-brc`

COMANDI

Il Comando `man`

Sintassi

`man [sezione] comando` — apre le pagine del manuale nel terminale.

Esempi: `man man`, `man cp`, `man ps`, `man chmod`. In alto a sinistra appare la sezione: `MAN(1)` indica sezione 1.

Le 10 Sezioni del Manuale

1. Comandi utente
2. Collegamenti del sistema
3. Funzioni del linguaggio C
4. Formati dei file
5. File di configurazione
6. Giochi
7. Varie
8. Comandi per l'amministrazione
9. Funzioni del kernel
10. Nuovi comandi

COMANDI

Navigare le Man Pages

Navigare

Frecce cursore, PagUp, PagDown

Cercare

Digitare `/parola` per cercare, poi `/` per cercare ancora

Uscire

Premere il tasto `q`

Esercizio: Provare ad usare alcune delle opzioni di `man` riportate nella sinossi completa.

COMANDI

Comandi `whatis`, `apropos`, `history`, `clear`

whatis

`whatis [opzioni] parolachiave` – cerca nel database whatis e fornisce una breve descrizione. Visualizza solo corrispondenze con parole intere.

apropos

`apropos stringa` – cerca nel database whatis. A differenza di whatis, visualizza tutte le corrispondenze. Es: `apropos keyboard`

history

Mostra i comandi eseguiti (di solito gli ultimi 500). Consente di ricercarli con i tasti freccia e rieseguirli.

clear

Rimuove il contenuto dello schermo. Gli input precedenti rimangono nello *scrollback buffer*.

COMANDI

Comandi `help` e `info`

`help`

Mostra una lista dei comandi shell integrati (built-in). `help comando shell` fornisce una descrizione del corrispondente comando.

Opzione `-h` / `--help`

Molti comandi accettano `-h` o `--help` per una breve descrizione sull'utilizzo del comando e delle sue opzioni.

`info comando`

Fornisce informazioni estese sul comando, con collegamenti che agevolano la navigazione nel manuale.

FILESYSTEM

Utenti, Filesystem e File

Organizzazione della memoria, struttura ad albero e gestione dei file in Linux

FILESYSTEM

Filesystem in Linux

Cos'è un Filesystem

Un'organizzazione di un'area di memoria basata sul concetto di **file** e **directory**. Induce una struttura gerarchica ad albero: solo le directory possono avere figli.

Tipi di File

File regolari: testi, dati, programmi sorgente, eseguibili

Directory: contengono altri file o directory

Device: dispositivi hardware visti come file

Pipe: scambio dati tra processi concorrenti

Link: riferimento ad un altro file o directory

Struttura del Filesystem Linux

Radice Unica

Linux ha un solo filesystem principale con radice `/` (directory *root*). Tutti i file e le directory sono contenuti, direttamente o indirettamente, in tale directory.

Nomi Case-Sensitive

All'interno della stessa directory non ci possono essere due file con lo stesso nome. Cambiare maiuscole/minuscole è sufficiente a distinguerli: `nomeFile` ≠ `nomefile`

Path Assoluto

Ogni file è raggiungibile dalla radice tramite un path assoluto: sequenza di directory separate da slash con slash come primo carattere. Es: `/home/utente1/dir1/dir3/file.png`

FILESYSTEM

Directory Principali di Ubuntu

/bin

Programmi basilari per la gestione del sistema, comandi base utilizzabili da qualsiasi utente

/boot

Immagini del kernel e file indispensabili al bootstrap del sistema

/dev

Periferiche hardware individuate sotto forma di file

/etc

File di configurazione del sistema (es. `/etc/apt` per i repository)

/home

Directory personali di tutti gli utenti del sistema

/lib

Librerie condivise del sistema

FILESYSTEM

Altre Directory Principali

/root

Home dell'amministratore di sistema, esplorabile solo con privilegi da super utente

/tmp

File temporanei

/usr

Contiene la maggior parte dei programmi installati: `/usr/bin` (eseguibili per tutti), `/usr/sbin` (solo amministratore), `/usr/share` (sfondi, icone, temi)

FILESYSTEM

Current Working Directory e Path

Comandi Fondamentali

`pwd` – mostra la current working directory (cwd)
`cd [path]` – cambia la cwd (senza path → home)
`..` – directory parent
`.` – la directory stessa

Path Assoluto vs Relativo

Assoluto: inizia con `/` o `~`, valido qualsiasi sia la cwd.

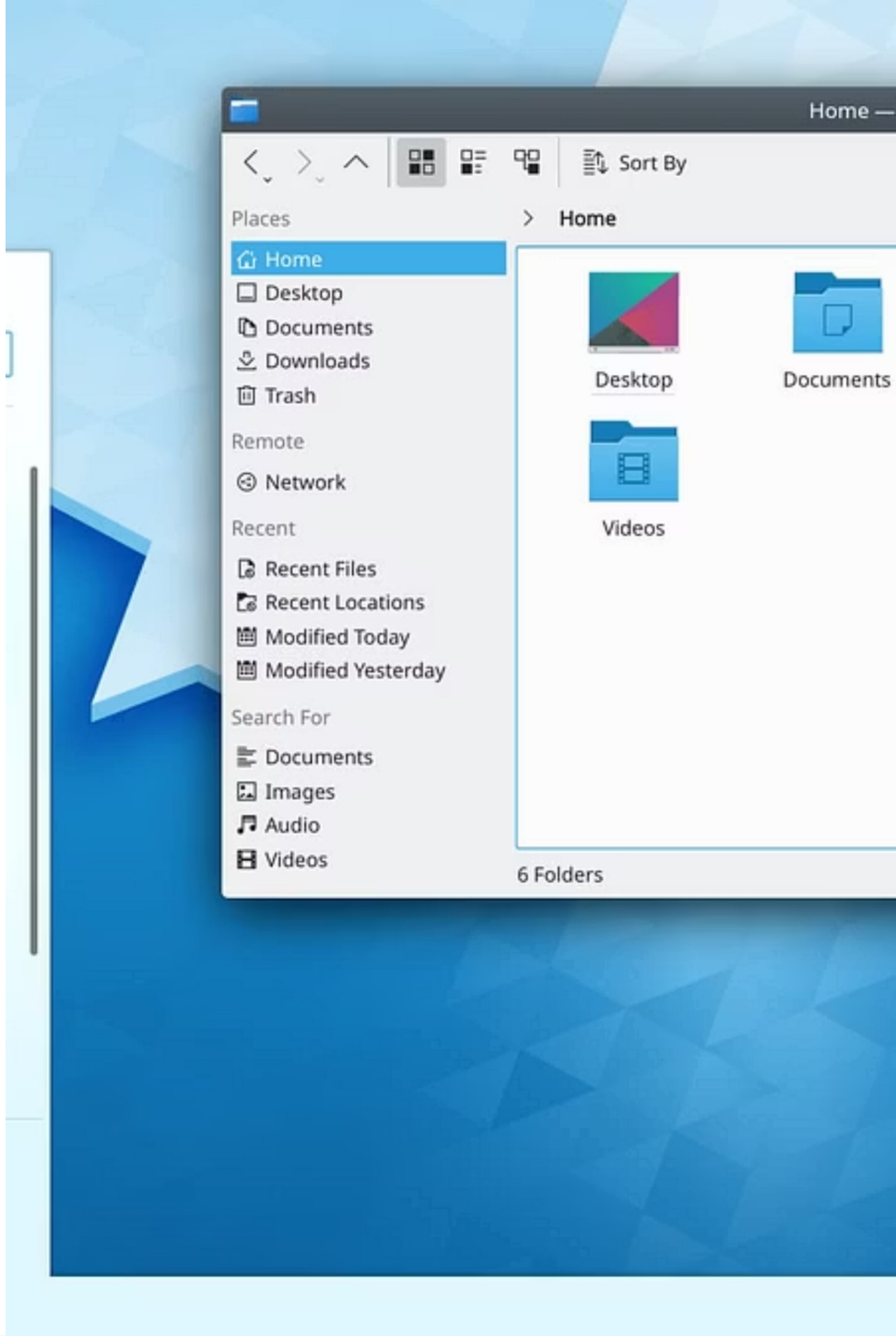
Relativo: non inizia con `/`, dipende dalla cwd corrente.

Esempio: se cwd è `/home/utente1`, il path relativo `dir1/dir3/file.png` equivale all'assoluto `/home/utente1/dir1/dir3/file.png`

Esercizio: Posizionarsi nella directory `/lib` e controllare come cambia il path nel prompt.

COMANDI FILE E DIRECTORY

Comandi per la Gestione di File e Directory



COMANDI

Comando `ls`

> ls [-la] [-R] [nomedir] – mostra il contenuto della directory `nomedir` (o della cwd se non specificata)

`ls -l`

Mostra informazioni dettagliate sul contenuto della directory

`ls -a`

Mostra i file *nascosti* (hidden), cioè i file con nomi che cominciano con il punto

`ls -R`

Mostra tutto il sottoalbero con radice in `nomedir`

`ls -s`

Mostra la dimensione dei file

COMANDI

Comandi `mkdir` e `touch`

`mkdir [-p] nomedir`

Crea la directory `nomedir` (vuota). Con l'opzione `-p`, se il path contiene più directory, le crea tutte (se non esistono).

Esempio: `mkdir -p dir11/dir13/dir15` crea `dir13` dentro `dir11` e `dir15` dentro `dir13`.

Esercizio: Provare senza l'opzione `-p` e osservare il risultato.

`touch nomefile`

Crea il file `nomefile` (vuoto). Per aprire e scrivere in un file si può usare un editor di testi:

`geany nomefile &` o `gedit nomefile &`
`nano nomefile` o `pico nomefile`
`vi nomefile` (per uscire: `:q` + invio)

Il carattere `&` serve per l'esecuzione in background.

COMANDI

Comando `xdg-open`

> `xdg-open nomefile|url` – apre un file o una URL con l'applicazione selezionata come *preferita*

File

Viene scelta l'applicazione preferita per il tipo di file

URL

Viene scelto il browser preferito

Opzioni

`-help`, `-manual`, `-version`

`xdg-open` fa parte del pacchetto `xdg-utils`, un insieme di tools che permettono di integrare varie applicazioni con la distribuzione Linux utilizzata.

COMANDI

Comando `cat`

> `cat [nomefile]` – scrive a schermo il contenuto di `nomefile`

Lettura di un File

Funziona bene solo se il file è di testo con codifica riconosciuta da `cat`; altrimenti scrive caratteri incomprensibili.

Lettura di Più File

`cat file1 file2 ... filen` stampa prima il contenuto del primo file, poi del secondo, ecc. L'output è la **concatenazione** del contenuto dei file.

Senza Argomenti

Resta in attesa: ripete quanto scritto finché non si preme `CTRL+d` (carattere EOF, *end-of-file*).

COMANDI

Creare File Non di Testo

Applicativi Dedicati

Per creare file non di testo occorre usare opportuni programmi:

- LibreOffice per documenti formattati (DOC, ODT), fogli di calcolo (XLS, ODS)

Comando: `libreoffice nomefile`

Attenzione: `nomefile` deve già esistere

Nota sull'Installazione

LibreOffice non è installato sul disco virtuale fornito e va installato usando diritti da amministratore. In alternativa, si può eseguire `libreoffice` senza argomenti e usare l'interfaccia grafica per creare un nuovo documento.

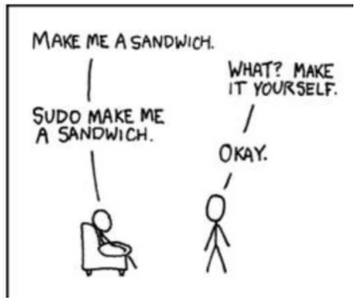
AMMINISTRAZIONE

Amministratore, Utente e `sudo`

Sudoer

In Ubuntu, l'utente creato a tempo di installazione è un **sudoer** — un utente con privilegi di amministratore di sistema — e appartiene al gruppo predefinito `sudo`.

Può eseguire comandi da superutente preponendo il comando `sudo` (*super user do*). Esempio: installare nuovi pacchetti.



COMANDI

Comando `tree`

> `tree [-a] [-L maxdepth] [-d] [-x] [nomedir]` – visualizza un intero albero di directory

Installazione

Potrebbe non essere installato: `sudo apt-get install tree`

Opzione `-L`

Limita la profondità dell'albero mostrato. Utile per alberi molto profondi.

Caratteri UTF-8

L'output contiene caratteri ASCII non-standard (UTF-8 a 3 bytes) per visualizzare la struttura dell'albero.

Interrompere

Se l'output sembra non finire mai, premere `CTRL+c`